

Deep Dive — Training Deep Networks (the craft)

Part 02 — 🧠 Deep Learning · 02-deep-learning/notes/02-training-deep-networks.md

Theory gets you a model that *should* train. This is about the one that *doesn't*.

1. The anatomy of a training loop

Every framework's loop, stripped to essentials:

```
for epoch in range(epochs):
    model.train()
    for batch in train_loader:
        optimizer.zero_grad(set_to_none=True) # 1. clear old gradients
        out = model(batch.x) # 2. forward
        loss = criterion(out, batch.y) # 3. loss
        loss.backward() # 4. backward (autograd)
        clip_grad_norm_(model.parameters(), 1.0) # 5. clip (optional but wise)
        optimizer.step() # 6. update
        scheduler.step() # 7. adjust LR
    validate(model, val_loader) # 8. eval, checkpoint best
```

Three bugs that account for most silent failures: 1. Forgetting `zero_grad()` → gradients accumulate across batches. Loss behaves strangely but doesn't crash. 2. Forgetting `model.eval()` → dropout and BatchNorm stay in training mode during validation. Your val score is noise. 3. Forgetting `torch.no_grad()` at eval → memory blows up, and it's slower for no reason.

2. The 15-minute debugging checklist

Run this **in order**. Stop at the first thing that's wrong.

- 1. Can you overfit 10 examples to ~zero loss?** If no, the bug is in your code — not your hyperparameters. This single test catches label misalignment, a detached graph, a frozen layer, and a wrong loss.
- 2. Is your loss at step 0 what theory predicts?** For K-way classification it should be $\ln(K)$ (2.303 for 10 classes). If it isn't, your output layer or loss is wrong.
- 3. Are the shapes right?** Print them. A silent broadcast of `(B,1)` against `(B,)` produces a `(B,B)` loss that trains to nonsense.
- 4. Is the data what you think?** Plot/print 10 raw inputs *after* the full transform pipeline. Check the label alignment.
- 5. Are gradients flowing?** Print `p.grad.norm()` per layer. All-zero early layers → vanishing gradient or a frozen module.

6. **Is the LR sane?** Try `1e-5`, `1e-4`, `1e-3`, `1e-2` for 100 steps each. Pick the largest that decreases smoothly.
 7. **Is normalization consistent** between train and inference? Train/serve skew lives here.
 8. **Is the metric computed on the right thing?** Accuracy on logits vs. probabilities vs. classes.
-

3. Vanishing & exploding gradients

Vanishing: repeated multiplication by small Jacobians drives gradients to zero; early layers never learn. Fixes: ReLU-family activations, He init, LayerNorm/BatchNorm, **residual connections**, LSTM gating.

Exploding: gradients blow up, loss $\rightarrow$ NaN. Fixes: **gradient clipping** (`clip_grad_norm_`, global norm 1.0 is the standard for transformers), lower LR, warmup, better init.

Diagnostic: log the global gradient norm every step. A healthy transformer run shows a norm that spikes during warmup and then settles into a stable band. A sudden $100\times$ spike usually precedes a loss divergence by a few dozen steps — and is often caused by one bad batch of data.

4. Hyperparameters, ranked by how much they matter

1. **Learning rate** — order of magnitude matters more than the exact value. Search on a log scale.
2. **Batch size** — set by memory; then scale LR with it (linear scaling rule + warmup).
3. **Architecture size** (width/depth) — bigger usually wins if you regularise and have data.
4. **Weight decay** — 0.01–0.1 for transformers; $1e-4$ for CNNs.
5. **Schedule** — warmup + cosine decay is the safe default.
6. **Dropout** — 0.0–0.1 for large models on large data; 0.3–0.5 for small models on small data.
7. Everything else — usually noise compared to 1–4.

Search strategy: random search beats grid search (Bergstra & Bengio 2012 — most dimensions don't matter, and grid wastes its budget on them). Bayesian optimisation (Optuna) beats random when each run is expensive. Always use the LR-range test first.

5. Making it fast (and affordable)

Technique	Speedup	Cost
Mixed precision (bf16/fp16 + fp32 master weights)	~2–3×	tiny accuracy risk; bf16 is safer than fp16
Gradient accumulation	simulates a big batch	more steps per update, same wall-clock
Gradient checkpointing	~40% less memory	~30% slower (recomputes activations)
<code>torch.compile</code>	1.3–2×	compile time; occasional graph breaks
Fused optimizers / <code>set_to_none=True</code>	a few %	none
DataLoader tuning (<code>num_workers</code> , <code>pin_memory</code> , <code>prefetch</code>)	often the biggest real win	none
Distributed Data Parallel (DDP)	~linear in GPUs	communication overhead
FSDP / ZeRO	trains models bigger than one GPU	complexity, sharding overhead

Check your GPU utilisation before optimising the model. If `nvidia-smi` shows 30% utilisation, your bottleneck is the data pipeline, and no amount of `torch.compile` will help.

6. Normalization placement (and why LLMs moved it)

- **Post-LN** (original transformer): `x + Sublayer(x)` then `LayerNorm`. Needs careful warmup; unstable at depth.
- **Pre-LN** (everything modern): `x + Sublayer(LayerNorm(x))`. Much more stable, trains without heroics.
- **RMSNorm**: drops the mean subtraction, keeps the scaling. Cheaper, works as well. Used by Llama-class models.

If you read a transformer diagram and the `LayerNorm` looks like it's in a strange place, it's this distinction.

7. Reproducibility

```
torch.manual_seed(s); np.random.seed(s); random.seed(s)
torch.backends.cudnn.deterministic = True      # slower
torch.use_deterministic_algorithms(True)      # will raise on nondeterministic ops
```

Also pin: library versions, dataset version/hash, and the exact config. Log everything to an experiment tracker (MLflow / Weights & Biases). **The run you can't reproduce didn't happen** — and under Module 12's governance requirements, it may not be deployable.

8. Checkpointing & model management

Save: model `state_dict`, optimizer state, scheduler state, epoch, RNG state, and the config. Keep the **best-by-validation** checkpoint, not the last. Version checkpoints with the code commit that made them. For the production version of this discipline, see Module 16 (model registry).

9. When deep learning is the wrong answer

- Tabular data with $< \sim 100k$ rows $\rightarrow$ gradient boosting (Module 01).
- You need per-prediction explanations for a regulator $\rightarrow$ start with an interpretable model (Module 12).
- Latency budget $< 10ms$ on CPU $\rightarrow$ a small model, or distillation.
- The task is genuinely solved by a rule or a lookup.

Choosing not to use deep learning is a senior move, not a junior one.

10. Practice checklist

- [] Overfit 10 examples on every new project before anything else.
- [] Log gradient norms, LR, and throughput — not just loss.
- [] Keep a `config.yaml`; never hard-code a hyperparameter.
- [] Evaluate on a fixed, never-tuned test split exactly once.
- [] Write down the failure modes you observed. That document becomes your eval suite (Module 15).